AI Physical Therapy & Rehabilitation Engine

Markerless Motion Capture and Kinematic Tracking System

1. Project Overview

The **AI Physical Therapy & Rehabilitation Engine** is an advanced, computer-vision-based application designed to facilitate at-home clinical physical therapy and fitness tracking. By leveraging markerless motion capture through Google MediaPipe and OpenCV, the system provides real-time biomechanical analysis, precise repetition counting, and immediate corrective audio-visual feedback.

This project was engineered to replace expensive, hardware-heavy motion capture systems with an accessible, highly accurate software solution. It is built entirely on **SOLID principles**, ensuring a highly decoupled, modular, and scalable architecture capable of supporting continuous expansions in clinical tracking methodologies.

2. Key Engineering Features

- **Real-time Kinematic Processing:** Utilizes OpenCV and MediaPipe Pose to extract continuous 3D spatial landmarks, achieving high-fidelity pose estimation at maximum framerates.
- **OS-Level Isolated Audio Feedback:** Completely eradicates Windows COM threading deadlocks by abandoning traditional Python TTS libraries in favor of a zero-latency, heavily isolated native PowerShell `System.Speech` subprocess architecture.
- **Dynamic Landmark Routing:** A highly flexible routing engine that dynamically re-allocates tracking focus between upper-body and lower-body skeletal nodes based on the currently selected exercise.
- **Adaptive FPS Regulation:** Intelligently analyzes the native frame rate of uploaded videos to normalize playback speed, preventing CPU fast-forwarding and ensuring smooth temporal tracking.
- **Model Warm-up & Zero Cold-Start Lag:** Pre-allocates MediaPipe's ML inference graphs in the background using initialized dummy tensor arrays, entirely eliminating initial-frame stutter and latency.
- **Automated Clinical Logging:** Tracks session progress, correct vs. incorrect repetitions, and peak joint extensions, safely exporting the clinical data to isolated session directories for long-term patient review.

3. System Architecture

The application is structured into discrete, highly specialized modules following the Single Responsibility Principle:

- **`app.py` (User Interface):** A modern, responsive desktop GUI built with `CustomTkinter`. It acts as the primary entry point, handling exercise con-

figuration, side selection, and safely managing the lifecycle of background tracking threads.

- `tracker.py` **(The Orchestrator):** The core engine loop. It handles video stream ingestion, MediaPipe inference, dynamic skeletal landmark extraction, and coordinates the flow of data between the mathematical, logic, and feedback modules.
- `kinematics.py` **(Biomechanical Mathematics):** A pure mathematics module responsible for calculating interior joint angles (via spatial coordinates and `math.atan2`) and computing the absolute elevation of limbs relative to the horizontal plane.
- `logic.py` **(Exercise State Machine):** The intelligent brain of the tracker. It evaluates the kinematic angles against clinical thresholds, operating a finite state machine (Resting -> Flexing -> Extending) to track range-of-motion, log successful repetitions, and instantly flag biomechanical errors (e.g., bent knees or dropped heels).
- `feedback.py` **(Audio-Visual Output):** Manages the real-time UI overlays (drawing angles, connection lines, and warning boxes) and governs the highly optimized background audio queue that fires immediate verbal instructions to the patient.
- `logger.py` **(Data Persistence):** A background utility that records real-time session metrics and safely exports them as JSON/CSV reports when the tracking session cleanly terminates.
- `requirements.txt`**:** The canonical list of project dependencies required to rebuild the tracking environment.

4. Supported Exercises & Biomechanics

The engine natively supports dynamic assessment for both lower-body and upper-body rehabilitation:

Heel Slides (Lower Body / Knee Mobility)

- **Tracked Landmarks:** Hip (23/24), Knee (25/26), Ankle (27/28)
- **Biomechanics:** Evaluates the interior knee angle. The repetition begins at a resting extension (>160°), requires deep flexion to hit the target (<110°), and must smoothly return to a full extension.
- **Error Detection:** Monitors the Y-axis coordinates of the ankle. If the foot elevates more than 45 pixels off the "bed" during the slide, it is instantly flagged as an improper lift error.

Straight Leg Raises (Lower Body / Hip Strength)

- **Tracked Landmarks:** Hip (23/24), Knee (25/26), Ankle (27/28)
- **Biomechanics:** Evaluates the absolute elevation angle of the entire leg relative to the horizontal plane. The patient must lift the leg from a flat position (<15°) to the target height (>45°) and lower it back down

smoothly.
- **Error Detection:** Strictly monitors the knee angle throughout the movement. If the knee bends (drops below 160°) during the lift, it triggers an immediate corrective warning to straighten the leg.

Biceps Curls (Upper Body / Arm Strength)

- **Tracked Landmarks:** Shoulder (11/12), Elbow (13/14), Wrist (15/16)
- **Biomechanics:** Evaluates the interior elbow angle. A correct repetition requires transitioning from a fully extended arm (>150°) to a tightly flexed curl (<45°) before returning to full extension.
- **Error Detection (Short-Rep Penalty):** If the arm begins to extend prematurely before reaching the target flexion depth, the system aborts the count, logs an incomplete rep error, and verbally instructs the user to complete the full range of motion.

5. Prerequisites & Installation

To run the AI Physical Therapy Engine locally, you will need **Python 3.8+** installed on a Windows machine.

1. **Clone the repository** to your local machine.
2. **Open a terminal** in the project's root directory.
3. (Optional but recommended) Create a virtual environment:

   ```
   python -m venv venv
   source venv/Scripts/activate
   ```

4. **Install the dependencies:**

   ```
   pip install -r requirements.txt
   ```

   (Core dependencies include `opencv-python`, `mediapipe`, `numpy`, and `customtkinter`.)

6. Usage Instructions

1. **Launch the Application:** Run the main UI file from your terminal:

   ```
   python app.py
   ```

2. **Configure the Session:**
   - Select your desired exercise from the **"Select Exercise"** dropdown (Heel Slides, Straight Leg Raise, or Biceps Curls).
   - Select the target limb from the **"Target Side"** dropdown (Left or Right).
3. **Start Tracking:**

- Click **"Open Live Camera"** to perform the exercise in real-time using your webcam. Ensure your full body is visible in the frame.
- *Alternatively*, click **"Upload Video"** to analyze a pre-recorded `.mp4` or `.avi` file.

4. **During the Session:**
    - Follow the verbal instructions and watch the on-screen overlays. The system will guide you through the correct range of motion.
    - Press the `ESC` key at any time to safely exit the tracking window and save your session data.
5. **Review Progress:**
    - Navigate to the newly generated `sessions/` folder in your project directory to view the exported clinical logs detailing your performance.